MODULE 21

API Observability and Monitoring

Explore how to gain deep visibility into the health, performance, and behavior of APIs in production.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 21 OVERVIEW

Build, observe, diagnose, and improve — observability is the foundation of production-ready APIs.

- This module explores how to gain deep visibility into APIs in production using logs, metrics, and traces, plus Spring Boot Actuator, alerting, and dashboards.

DURATION & LAB

Theory

Three pillars, Actuator, tracing, alerting, dashboards.

Lab

Add Actuator health checks and Micrometer metrics.

Scenario

Instrument, visualize, alert, and analyze issues.

BUILD

Metrics, logs, and traces via Actuator + Micrometer

OBSERVE & DIAGNOSE

Dashboards, alerting, distributed tracing

THE PAYOFF

Faster detection, faster root cause, higher reliability

KEY TAKEAWAY Good observability leads to faster insights and better outcomes — you can't improve what you can't see.

WHY OBSERVABILITY MATTERS

In modern distributed systems, issues are inevitable — observability gives you deep visibility to find and fix them fast.

Faster Issue Detection

Root Cause Analysis

Improved Reliability

Better User Experience

Data-Driven Decisions

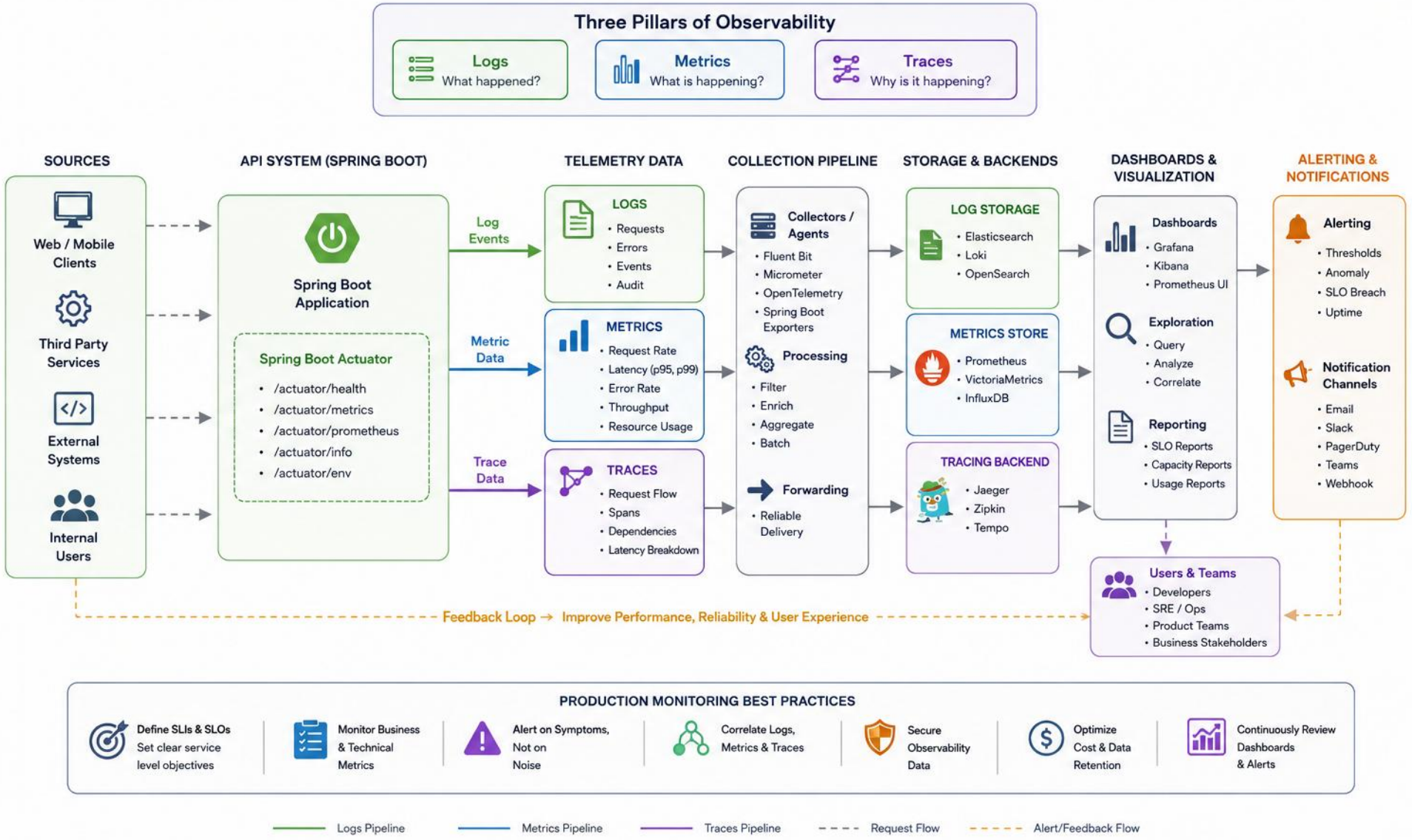
Business Impact

Impact chain: Operational Impact (efficient ops) + Engineering Impact (faster debugging) + Customer Impact (reliable APIs) roll up to Business Impact — lower costs, better reliability, greater customer trust.

KEY TAKEAWAY You can't improve what you can't see — observability turns invisible failures into actionable insights.

API OBSERVABILITY

Gain full visibility into API health, performance, and user experience



MONITORING VS OBSERVABILITY

Monitoring watches defined signals and conditions. Observability is a system property: how well internal behavior can be inferred from emitted telemetry.

MONITORING

- Practice of watching defined signals, dashboards, and conditions.
- Key question: "Are our defined service objectives and known conditions healthy?"
- Predefined metrics, thresholds, and alerts.
- One of the ways teams USE observability data.

VS

OBSERVABILITY

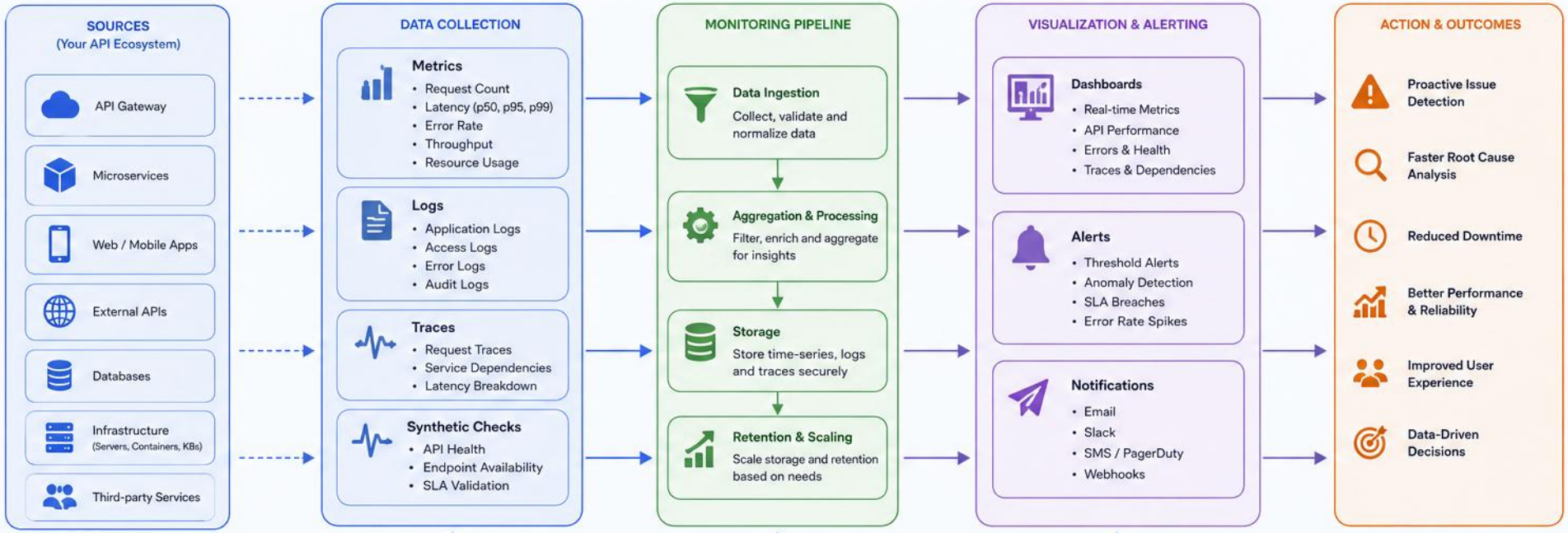
- System property: how well internal behavior can be inferred from telemetry.
- Key question: "Do we have enough telemetry and context to investigate unexpected behavior?"
- Dynamic exploration using logs, metrics, and traces together.
- Instrumentation → telemetry → monitoring, alerting, investigation, capacity planning.

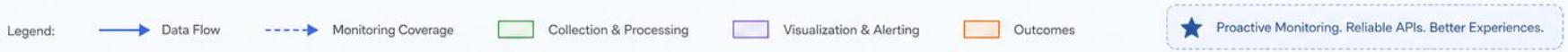
Monitoring is one way teams use observability data — not a competing approach. Instrumentation produces telemetry; telemetry feeds monitoring, alerting, investigation, and capacity planning.

KEY TAKEAWAY Monitoring watches defined signals for known conditions. Observability is whether your telemetry can answer the unknown questions.

API Monitoring Architecture

Real-time Visibility. Proactive Alerts. Reliable APIs.





THREE PILLARS OF OBSERVABILITY

Logs, metrics, and traces are the three core telemetry signals taught in this module. Mature observability platforms may also add profiles, events, and user-experience telemetry.

LOGS

What: Discrete events that happen in your system.

Answers: What happened?

Best for: Debugging, auditing, troubleshooting.

METRICS

What: Aggregated numerical data on health and performance.

Answers: How is the system performing?

Best for: Alerting, dashboards, capacity planning.

TRACES

What: End-to-end request flows across services.

Answers: Where is the issue happening?

Best for: Root cause analysis, latency tracking.

KEY TAKEAWAY Logs, metrics, and traces provide complementary evidence — consistent instrumentation and operational practice turn them into insight.

The Three Pillars of Observability

Observability is built on 3 types of telemetry data that help you understand your system.

System / Application — Generates telemetry → **Collect** — Gather data → **Analyze** — Correlate & analyze → **Insight** — Understand & act

LOGS

What happened?

Discrete events recorded at a specific point in time.
Best for understanding detailed behavior.

Examples

- Error messages
- Exception stack traces
- User activities
- Audit events

```
{
  "timestamp": "2026-07-17T10:15:302",
  "level": "ERROD",
  "service": "order-service",
  "message": "Payment failed",
  "orderId": "12345",
  "userId": "u789"
}
```

Best used for

- Investigating issues
- Root cause analysis
- Auditing and compliance

Analogy

Logs are like detailed news articles about what happened.

METRICS

What is happening?

Aggregated numerical measurements over time.
Best for understanding trends and system health.

Examples

- Request rate (RPS)
- Error rate (%)
- Latency (p50, p95, p99)
- CPU / Memory usage
- Throughput

Best used for

- Monitoring system health
- Detecting anomalies
- Capacity planning & alerting

Analogy

Metrics are like dashboards showing the speed and health of your car.

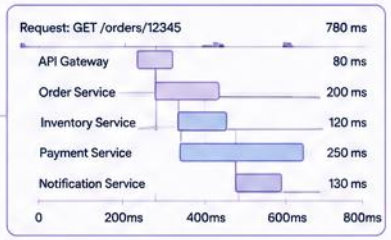
TRACES

Why is it happening?

Requests flow through your system and across services.
Best for understanding dependencies and latency.

Examples

- End-to-end request
- Spans across services
- Latency breakdown
- Dependencies



Best used for

- Understanding request flow
- Finding performance bottlenecks
- Analyzing service dependencies

Analogy

Traces are like GPS showing the full journey and where delays occur.

Stronger Together

Logs tell you what happened. + Metrics show you the impact and trends. + Traces explain why it happened and where. = Full observability gives you the complete picture to understand and improve your system.

Observability is not about more data, it's about the right data, correlated to answer the right questions.

LOGS

Module 20 covered structured logs, correlation IDs, privacy, and centralized logging in depth. Here, logs are correlated with metrics and traces during diagnosis.

EXAMPLE LOG ENTRY (JSON)

```
{
  "timestamp": "2025-05-20T10:15:24.123Z",
  "level": "INFO", "service": "order-service",
  "traceId": "a1b2c3d4e5f67890", "spanId": "b1c2d3e4f5678901",
  "message": "Order created successfully", "userId": "u12345",
  "orderId": "ord-98765", "status": "SUCCESS", "durationMs": 142
}
```

CROSS-SIGNAL WORKFLOW

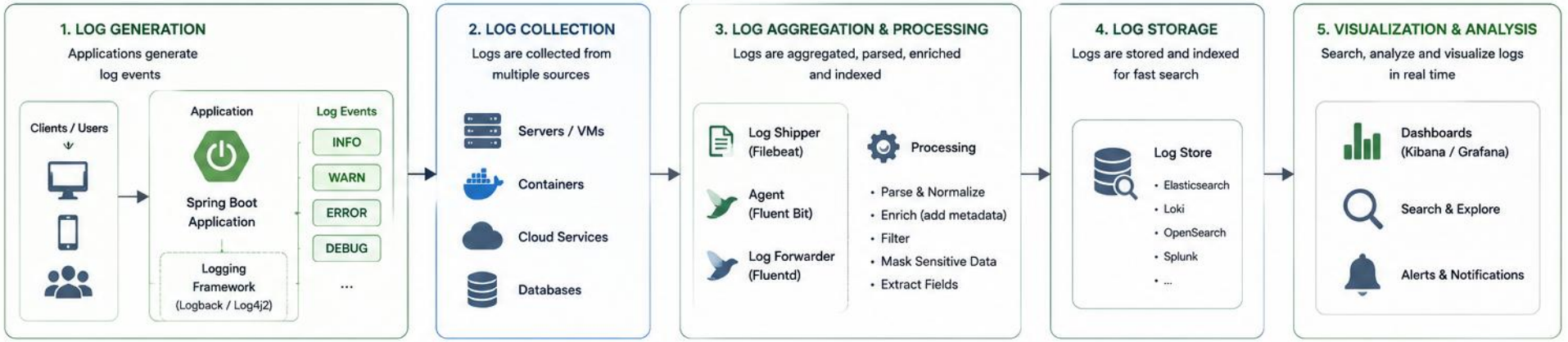
- Workflow: Metric alert -> trace ID -> related structured logs.
- Include only approved, non-sensitive identifiers in logs. Do not log names, emails, tokens, session secrets, or regulated data.

Log levels (low to high severity): TRACE -> DEBUG -> INFO -> WARN -> ERROR. Typical flow: Application -> Log Collector (Filebeat/Fluentd) -> Log Store (Elasticsearch/CloudWatch) -> Visualization (Kibana/Grafana).

KEY TAKEAWAY Logs still matter, but in this module they are correlated with metrics and traces, not learned again from scratch.

LOGS

Record discrete events that help you understand what happened in your system.



WHAT DOES A LOG CONTAIN?

```
{
  "timestamp": "2026-07-17T10:15:30.1232",
  "level": "ERROR",
  "service": "order-service",
  "traceId": "4f5a2c6b1e3d4f9a",
  "spanId": "a1b2c3d4e5f6g7h8",
  "thread": "http-nio-8080-exec-3",
  "logger": "com.example.order.Controller",
  "message": "Payment failed for order 12345",
  "userId": "u789",
  "ip": "192.168.1.10",
  "exception": "java.lang.Exception:
                Insufficient funds",
  "stackTrace": "...",
}
```

timestamp	When the event occurred
level	Severity (INFO, WARN, ERROR, DEBUG)
service	Source service/application
traceId	Correlates logs with a trace
spanId	Correlates with a span (optional)
thread	Thread handling the request
logger	Class or component that logged
message	Human readable log message
userId/ip	Context about the user/request
exception	Error type if any
stackTrace	Detailed stack trace for errors

LOG LEVELS

TRACE	Finest level of logs for debugging
DEBUG	Detailed information for developers
INFO	General operational information
WARN	Something unexpected happened but system is running
ERROR	A failure occurred in a request/ operation
FATAL	Severe error – may cause system shutdown

BEST PRACTICES

- Use appropriate log levels
- Log meaningful messages, not just codes
- Include context (userId, requestId, traceId, etc.)
- Use structured logging (JSON)
- Avoid logging sensitive data (passwords, tokens, PII)
- Centralize logs
- Set retention & rotation policies
- Monitor logs and set up alerts

COMMON USE CASES

- Troubleshooting Issues
- Root Cause Analysis
- Security Auditing & Compliance
- User Activity Audit
- Business Insights & Monitoring

KEY TAKEAWAY

Logs tell you what happened. They provide the facts you need to understand, troubleshoot and improve your system.

METRICS

Metrics are numerical measurements collected over time, giving quantifiable insight into health, performance, and behavior.

METRIC	DESCRIPTION	TYPE	EXAMPLE
`HTTP Request Rate`	Number of requests per second	Counter	125 req/s
`Response Time (p95)`	95th percentile of API response time	Derived (timer/histogram)	480 ms
`Error Rate`	Percentage of failed requests	Derived (counters)	1.23 %
`CPU / Memory Usage`	Resource utilization	Gauge	62% / 1.4 GB

RED (RATE, ERRORS, DURATION) + USE (UTILIZATION, SATURATION, ERRORS) — DISTINCT FROM GOLDEN SIGNALS

RED Method

Rate, Errors, Duration — for request-driven services.

USE Method

Utilization, Saturation, Errors — for infrastructure/resources.

Drive Alerting

Use metric thresholds to trigger alerts and automation.

Metric types: Counter (increases only; resets on restart — query rates over time, not raw restart-to-restart values), Gauge (up/down), Timer/Histogram (duration distribution; preferred for cross-instance percentiles), Summary (client quantiles — may not aggregate across replicas).
Business examples: creation success/rejection-by-reason, lookup success/not-found, processing duration, readiness state. Avoid as labels: customer ID, email, request ID, trace ID, raw error text.
Low-cardinality labels: result=success, operation=create, reason=duplicate, route=/customers/{id}. Dangerous: customerId, traceId, errorMessage, full URI — risk: excess series, memory, cost, instability.

KEY TAKEAWAY Instrument counters, timers, and gauges correctly, keep labels low-cardinality, and combine metrics with logs and traces.

MICROMETER CUSTOM METRICS

Register Counters and Timers directly on the MeterRegistry to expose RED-method signals for each operation.

EXAMPLE: COUNTER AND TIMER METRICS (JAVA)

```
@Component
class CustomerMetrics {
    private final MeterRegistry registry;
    CustomerMetrics(MeterRegistry registry) { this.registry = registry; }
    void recordCreate(boolean success, long durationMs) {
        registry.counter("crm.customer.operations",
            "operation", "create", "result", success ? "success" : "failure").increment();
        registry.timer("crm.customer.operation.duration", "operation", "create")
            .record(Duration.ofMillis(durationMs));
    }
}
```

KEY POINTS

- Counters only increase — track completed events like successful or failed creates.
- Timers record both count and duration; use Gauge separately for point-in-time values like queue depth.

KEY TAKEAWAY Register meters once per component and reuse them — creating a new Counter or Timer per request adds overhead.

TRY IT YOURSELF — EXERCISE 1: CARDINALITY ANTI-PATTERNS

Reinforces: the low-cardinality label guidance from the METRICS slide you just saw.

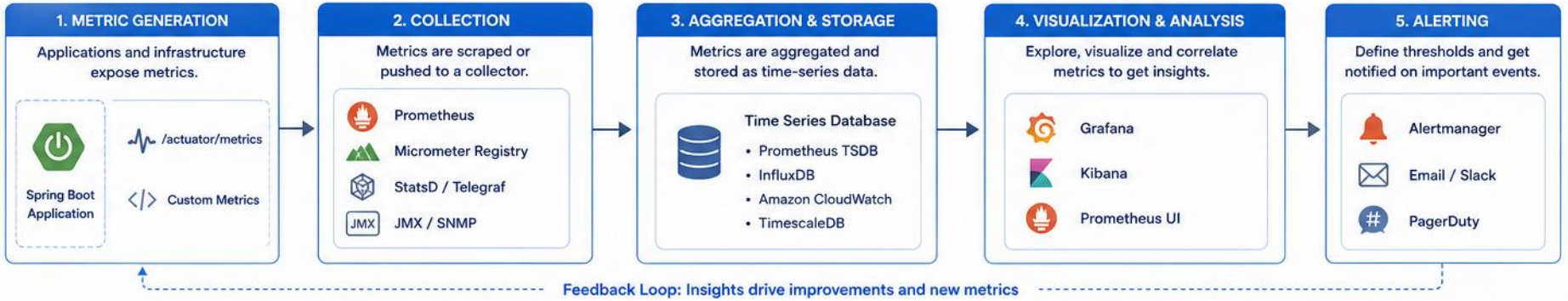
STEP	WHAT TO DO
Goal	Recreate the label do/don't table; add one URL-shaped anti-pattern
File	lab21-cardinality-table.md (in examples/module-21-exercises/)
Do / Don't	outcome=success|failure OK; customerId=CUS-1001 NO — high cardinality
Anti-pattern	Raw URL with an id (/customers/CUS-1001) is as risky as a customerId label
Good metric	customer_create_failure_total with reason=validation|conflict
Reference	exercises/exercise-01-cardinality-antipatterns.md

```
counter("customer_create_total")
  .tag("customerId", customerId)   // BAD -- one series per customer, forever
counter("customer_create_failure_total")
  .tag("reason", "validation")     // GOOD -- reason has ~3 fixed values
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-cardinality-antipatterns.md.

METRICS

Quantitative measurements over time that help you understand what is happening.



TYPES OF METRICS

Counter
A metric that only increases. Reset on restart.
Examples
- http_requests_total
- errors_total

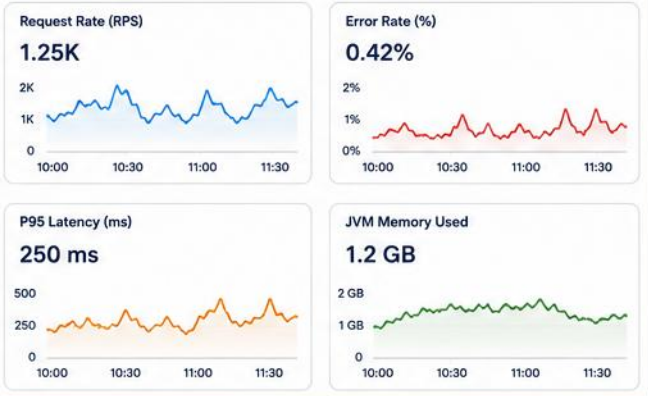
Gauge
A metric that can go up or down.
Examples
- jvm_memory_used_bytes
- cpu_usage_percent

Histogram / Summary
Records a distribution of values.
Examples
- http_request_duration_seconds
- db_query_duration_seconds

COMMON METRICS (EXAMPLES)

Category	Metric	Description	Type
HTTP	http_requests_total	Total number of HTTP requests	Counter
	http_request_duration_seconds	Request duration	Histogram
	http_server_requests_seconds_max	Max request time	Gauge
JVM	jvm_memory_used_bytes	JVM memory used	Gauge
	jvm_threads_live	Live threads	Gauge
	jvm_gc_pause_seconds_total	Total GC pause time	Counter
SYSTEM	process_cpu_usage_percent	CPU usage %	Gauge
	system_load_average	System load average	Gauge
	disk_usage_bytes	Disk space used	Gauge

METRIC VISUALIZATION EXAMPLE



BEST PRACTICES

- Use consistent naming conventions
- Include meaningful tags/labels (e.g., service, env, region)
- Monitor SLIs and business metrics
- Define thresholds based on SLOs
- Avoid high-cardinality metrics
- Review and remove unused metrics
- Retain metrics for the right duration

HOW METRICS HELP

Observe
Collect measurements continuously

Analyze
Identify patterns, trends & anomalies

Understand
Gain insights into system health & performance

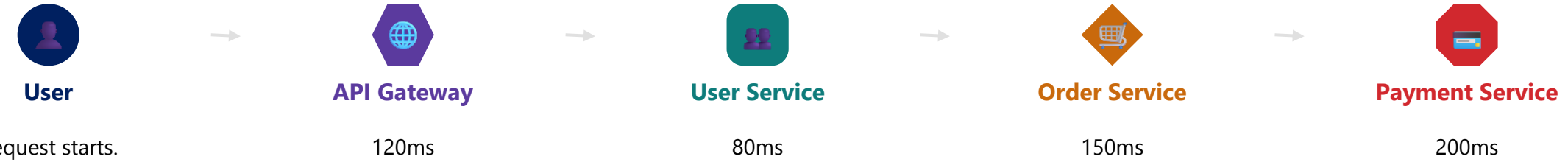
Act
Take action & improve the system

USE CASES

- Monitor system health & performance
- Capacity planning & resource optimization
- Detect anomalies & potential issues
- Measure SLIs and ensure SLOs
- Business & user experience monitoring

DISTRIBUTED TRACING FUNDAMENTALS

A trace shows how one request flows through multiple services. Spans capture timing and metadata, and context propagates across service boundaries.



A trace is made up of spans, each with a start time, end time, duration, and metadata. End-to-end latency follows the critical path — child-span durations may overlap, so do not always add every span duration together.

TRACE, SPAN & CONTEXT PROPAGATION

- Trace = the end-to-end journey of a request. Span = one unit of work within it; child spans reference a parent span ID (parent-child hierarchy).
- Continue an incoming W3C trace context when present; otherwise create a new trace at the system boundary — not at every entry point.
- Do not attach request bodies, tokens, raw SQL with personal values, or full customer records to spans — use route template, HTTP status, operation name, and safe error type instead.

Roles: OpenTelemetry provides instrumentation and context; the OpenTelemetry Collector handles collection/export; Jaeger, Zipkin, and vendor APM platforms serve as backends. Sampling: head sampling, tail sampling, and error-biased retention balance cost — not every request has a stored trace.

KEY TAKEAWAY Traces reveal latency and root cause across services — protect sensitive span data and sample wisely.

INSTRUMENTING TRACES IN CODE

Micrometer Tracing creates spans automatically at instrumented boundaries; use annotations or the Tracer API to add custom spans and tags.

EXAMPLE: CUSTOM SPAN WITH @NEWSPAN (JAVA)

```
@Service
class OrderService {
    private final Tracer tracer;
    OrderService(Tracer tracer) { this.tracer = tracer; }
    @NewSpan("order-place")
    public Order placeOrder(@SpanTag("customerId") String customerId,
                            OrderRequest request) {
        tracer.currentSpan().tag("order.itemCount", String.valueOf(request.items().size()));
        return orderRepository.save(toOrder(customerId, request));
    }
}
```

KEY POINTS

- @NewSpan starts a span at the annotated method; @SpanTag adds low-cardinality context to it.
- Manual spans via Tracer suit code Micrometer doesn't auto-instrument, like custom business logic.

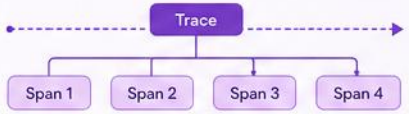
KEY TAKEAWAY Reach for @NewSpan and @SpanTag first — hand-roll spans only for logic Micrometer can't auto-instrument.

TRACES

Follow a request across services. Understand every step. Fix faster.

WHAT ARE TRACES?

Traces show the path of a request as it travels through multiple services. Each trace is made up of spans that represent individual operations.



One request. Many spans. Full visibility.

KEY CONCEPTS

- **Trace ID** – Unique identifier for a request across services
- **Span** – A unit of work within a trace (e.g., database call)
- **Parent Span** – The operation that triggered another
- **Attributes** – Key-value data about the operation
- **Duration** – Time taken by a span

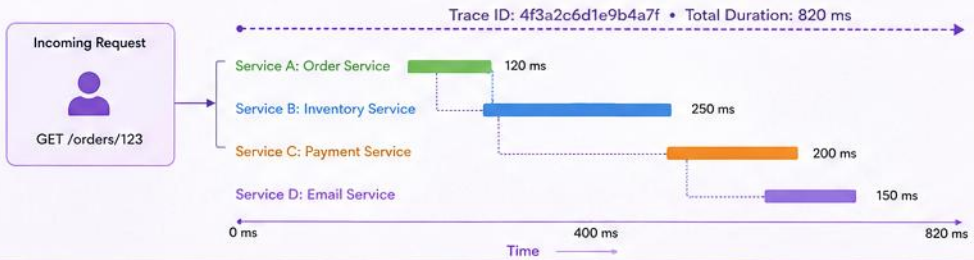
HOW TRACES WORK

Each service creates a span and adds context so the entire request can be stitched together.

BENEFITS

- End-to-end visibility across services
- Identify slowdowns and bottlenecks
- Debug errors across distributed systems
- Understand dependencies and flows
- Improve performance and reliability

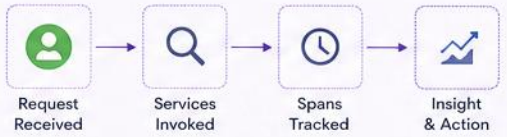
DISTRIBUTED TRACE EXAMPLE



SPAN LEGEND

- Service A 120 ms
- Service B 250 ms
- Service C 200 ms
- Service D 150 ms
- Parent → Child

TRACE IN ACTION



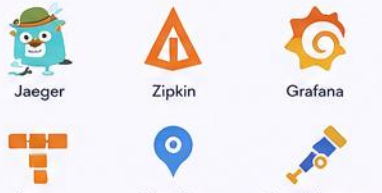
SPAN DETAILS (EXAMPLE)

Span ID	Operation	Service	Start Time	Duration	Status	Attributes (Key→Value)
a1b2c3d	GET /orders/123	Order Service	10:00:00.100	120 ms	✓	http.method=GET http.url=/orders/123
d4e5f6g	GET /inventory/123	Inventory Service	10:00:00.220	250 ms	✓	db.system=postgres rows=1
h7i8j9k	POST /payments	Payment Service	10:00:00.470	200 ms	✓	provider=stripe status=success
l1m2n3o	POST /email/send	Email Service	10:00:00.670	150 ms	✓	provider=smtp to=user@example.com

COMMON SPAN ATTRIBUTES

- http.method – Request method
- http.url – Request URL
- status.code – HTTP status code
- db.system – Database type
- peer.service – Calling service
- error – Error details (if any)
- user.id – User identifier
- env – Environment (prod, staging)

TOOLS & INTEGRATIONS



BEST PRACTICES

- Instrument all critical services
- Propagate trace context (W3C Trace Context)
- Sample intelligently (adaptive sampling)
- Capture meaningful attributes
- Monitor and alert on slow traces and errors
- Review traces regularly for optimization

SPRING BOOT ACTUATOR OVERVIEW

Actuator provides low-effort operational endpoints and baseline instrumentation. Production monitoring still requires configuration, security, and operational design.

ENDPOINT	PURPOSE
`/actuator/health`	Health status of the application
`/actuator/info`	Application information
`/actuator/metrics`	Application metrics
`/actuator/prometheus`	Metrics in Prometheus format
`/actuator/env`	Environment properties

ENABLING ACTUATOR (application.yml)

```
<dependency><artifactId>spring-boot-starter-actuator</artifactId></dependency>
<dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</artifactId></dependency>
management:
  endpoints: {web: {exposure: {include: health,info,metrics,prometheus}}}
  endpoint: {health: {probes: {enabled: true}, show-details: when_authorized}}
```

RESTRICTED DIAGNOSTIC ENDPOINTS — /actuator/env, /beans, /mappings, /loggers, /threaddump: never expose publicly; require authN/authZ, least privilege, and audited access (a separate management port/network is ideal). exposure.include controls WEB EXPOSURE; security config decides WHO can access.

KEY TAKEAWAY Actuator is a low-effort start, not a finished solution — secure, expose deliberately, and add real instrumentation.

TRY IT YOURSELF — EXERCISE 2: ACTUATOR ALLOW-LIST

Reinforces: the restricted diagnostic endpoints you just saw on the Actuator overview.

STEP	WHAT TO DO
Goal	Draft which Actuator endpoints ship exposed vs. locked down
File	actuator-allowlist.md (in examples/module-21-exercises/)
Candidates	health, info, metrics, prometheus, env, beans, mappings, loggers
Allow-list	Expose health (+info); lock env, beans, configprops, loggers
Auth note	exposure.include controls reachability -- scrapes still need network/auth
Reference	exercises/exercise-02-actuator-allowlist.md

```
management:
  endpoints: {web: {exposure: {include: health,info,metrics,prometheus}}}
  endpoint: {health: {probes: {enabled: true}, show-details: when_authorized}}
# never public: env, beans, mappings, loggers, threaddump
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-actuator-allowlist.md.

Spring Boot Actuator

Production-ready features to monitor and manage your Spring Boot applications.

Spring Boot Actuator provides built-in endpoints to monitor application health, metrics, info, and more.
It helps you operate, monitor, and manage your application in production.

1. ENABLE ACTUATOR

Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

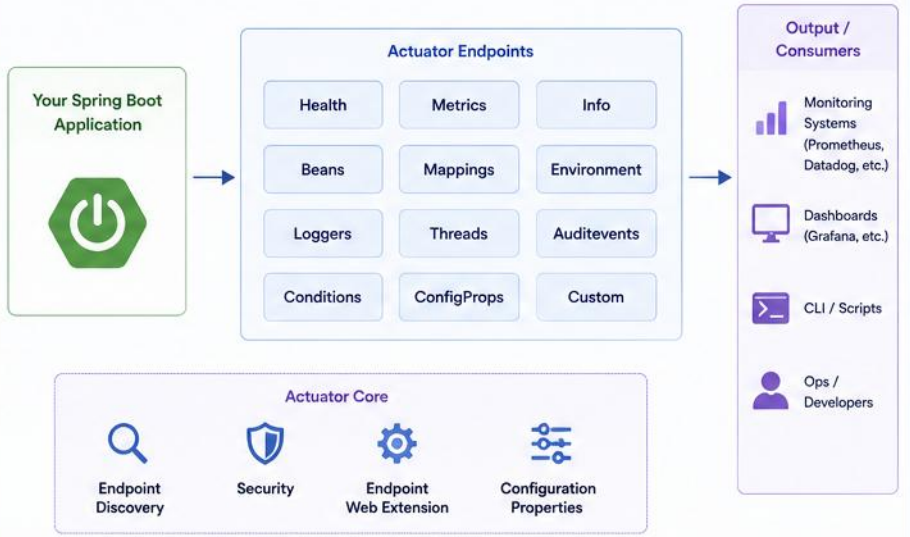
Gradle

```
implementation
  'org.springframework.boot:spring-boot-
  starter-actuator'
```

application.properties / application.yml

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=when_authorized
```

2. ACTUATOR ARCHITECTURE



3. COMMON ACTUATOR ENDPOINTS

Endpoint	Path	Description
Health	/actuator/health	Application health status
Metrics	/actuator/metrics	Application metrics (counters, gauges, etc.)
Info	/actuator/info	Application information
Beans	/actuator/beans	List of Spring beans
Mappings	/actuator/mappings	Request mapping details
Environment	/actuator/env	Environment properties
Config Props	/actuator/configprops	Configuration properties
Loggers	/actuator/loggers	View and modify logging levels
Threads	/actuator/threaddump	Thread dump
Auditevents	/actuator/auditevents	Security audit events
Shutdown*	/actuator/shutdown	Graceful shutdown (disabled by default)

Security Note: Actuator endpoints can expose sensitive information.
Always secure them in production using authentication and authorization.

Base Path (default)
/actuator

Example:
http://localhost:8080/actuator/health

4. HOW IT WORKS

1. Request
Client requests an actuator endpoint

2. Endpoint Handler
Actuator maps the request to the appropriate endpoint

3. Data Collection
Gathers data from application context, metrics registry, etc.

4. Response
Returns data in JSON (or other supported format)

5. CUSTOMIZATION

- **Include/Exclude Endpoints**
  `management.endpoints.web.exposure.include=health,info,metrics`
- **Change Base Path**
  `management.endpoints.web.base-path=/manage`
- **Customize Health Indicators**
  Create custom HealthIndicator beans
- **Tagging & Additional Info**
  Expose custom info and metrics

6. BENEFITS

- Built-in monitoring and management
- Production-ready insights
- Seamless integration with monitoring tools
- Easy to extend and customize
- Helps in faster troubleshooting
- Improves application reliability

INTEGRATION ECOSYSTEM

- Prometheus
- Grafana
- Elastic Stack
- Datadog

* The /actuator/shutdown endpoint is disabled by default and should only be enabled in controlled environments.

HEALTH ENDPOINTS

Health endpoints provide real-time status about your application and its dependencies — liveness and readiness answer different questions.

ENDPOINT	DESCRIPTION	TYPICAL USE
`/actuator/health`	Overall health status	Basic health check
`/actuator/health/liveness`	Is the application alive?	Kubernetes liveness probe
`/actuator/health/readiness`	Is it ready to serve traffic?	Kubernetes readiness probe
`/actuator/health/db`	Database health	Check DB connectivity

EXAMPLE RESPONSE (AUTHORIZED VIEW)

```
{ "status": "UP",
  "components": { "db": { "status": "UP" },
    "diskSpace": { "status": "UP" } },
  "groups": ["liveness","readiness"] }
```

BEST PRACTICE

- Liveness: is the process stuck/irrecoverable and should it restart? Avoid checking external dependencies that may recover on their own.
- Readiness: can this instance serve traffic now? A DB outage should usually fail readiness, not liveness.

Health probes signal orchestrators and load balancers to route or recover traffic — the endpoint itself does not perform recovery. Component subpaths (e.g. /health/db) vary by Spring Boot version; verify for yours.
Custom readiness checks must stay fast and bounded: no expensive remote calls, long database queries, or full business transactions on every probe.

KEY TAKEAWAY Liveness and readiness answer different questions — health signals drive orchestrator decisions, they don't make them.

CUSTOM HEALTH INDICATORS

Implement HealthIndicator to add application-specific checks — like the CrmReadinessIndicator built in this module's lab.

EXAMPLE: CUSTOM HEALTHINDICATOR (JAVA)

```
@Component("crmReadiness")
record CrmReadinessIndicator(CustomerRepository customers) implements HealthIndicator {
    public Health health() {
        try {
            customers.count();
            return Health.up().build();
        } catch (DataAccessException ex) {
            return Health.down(ex).build();
        }
    }
}
```

KEY POINTS

- Spring Boot auto-registers any HealthIndicator bean and aggregates it into /actuator/health.
- Add it to the readiness group so orchestrators stop routing traffic without restarting the pod.

KEY TAKEAWAY A custom HealthIndicator lets liveness and readiness reflect real dependencies — not just 'the JVM is running.'

TRY IT YOURSELF — EXERCISE 3: LIVENESS VS READINESS

Reinforces: the liveness vs. readiness distinction from the Health Endpoints slide you just saw.

STEP	WHAT TO DO
Goal	Explain when Kubernetes should restart the CRM pod vs. just stop routing traffic
File	lab21-probes.md (in examples/module-21-exercises/)
Liveness	Process stuck/deadlocked → restart (CRM: deadlocked request threads)
Readiness	Dependency down (DB) → not ready; keep process, drop from load balancer
Wrong mix	Don't kill the pod on every DB blip — readiness can gate traffic instead
Reference	exercises/exercise-03-liveness-vs-readiness.md

```
$ curl localhost:8080/actuator/health/liveness
{ "status": "UP" }                  // process is fine -- do NOT restart
$ curl localhost:8080/actuator/health/readiness
{ "status": "OUT_OF_SERVICE" }  // DB down -- stop routing, keep process alive
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-liveness-vs-readiness.md.

METRICS COLLECTION

Micrometer is a metrics and observation instrumentation facade that Spring Boot uses — much more than just an export library.

MICROMETER REGISTRIES (EXPORTS) — PROMETHEUS IS ONE OPTION

Prometheus

Pull-based: Prometheus scrapes /actuator/prometheus on an interval.

Datadog / New Relic

Push-based: the application/agent exports telemetry to the vendor.

CloudWatch

Push-based, AWS-native monitoring and export.

```
Timer.Sample s = Timer.start(registry); try { return service.createCustomer(req); } finally {
s.stop(Timer.builder("crm.customer.operation").tag("operation","create").register(registry)); }
Use finally so duration records even on failure. Prefer pre-registering meters in a dedicated component (e.g. CustomerMetrics) rather than
scattering builders through business-logic code.
```

KEY TAKEAWAY Micrometer is Spring Boot's instrumentation facade — Prometheus is one export option, not the only one.

TRY IT YOURSELF — EXERCISE 4: METRIC SKETCH (STARTER)

Reinforces: the Micrometer counters and timers you just saw — now sketch your own for CRM create.

STEP	WHAT TO DO
Goal	FILL-IN-THE-BLANK STARTER — complete 6 blanks, not a from-scratch design
File	lab21-metric-todos.md (in examples/module-21-exercises/)
Fill in	Success/failure counters, forbidden label, alert name, threshold, action
Expected	create_success_total, create_failure_total, customerId forbidden
Alert narrative	Page on sustained failures; correlate via logs, never via a label
Reference	exercises/exercise-04-fill-metric-sketch-todos.md (starter)

```
Success counter: _____
Failure counter: _____
Forbidden label: _____
Alert name: _____     Threshold: _____
First responder action: _____
```

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-fill-metric-sketch-todos.md.

SERVICE PERFORMANCE SIGNALS

Track request-level signals (RED) plus resource saturation using percentiles, not averages, to catch degradation before users notice.

Response Time

Throughput

Error Rate

CPU / Memory

PERCENTILES, SLOs & DIAGNOSIS

- Prefer percentiles (p50, p95, p99) over averages — watch request volume alongside latency; averages hide outliers.
- SLI = measured indicator (e.g. successful-request %); SLO = target (e.g. 99.9% over 30 days); SLA = contractual commitment; error budget = allowed unreliability.
- Diagnosis example: p95 latency rises while error rate is normal and CPU saturation climbs — investigate the saturated dependency before changing code.
- Correlate metrics with logs and traces to confirm root cause quickly.

Example alert: burn rate shows the error budget is being consumed too fast — page before the SLO is breached, not after. Common tools: Prometheus, Grafana, Actuator, Micrometer, Datadog, CloudWatch.

KEY TAKEAWAY Measure with percentiles, define SLOs, and correlate signals — performance monitoring is proactive, not reactive.

CONFIGURING TIMER PERCENTILES

Publish p50/p95/p99 for request timers instead of relying on client-side averages that hide outliers.

EXAMPLE: PERCENTILE CONFIGURATION (APPLICATION.YML)

```
management:
  metrics:
    distribution:
      percentiles:
        http.server.requests: 0.5, 0.95, 0.99
      percentiles-histogram:
        http.server.requests: true
      slo:
        http.server.requests: 100ms, 200ms, 400ms
```

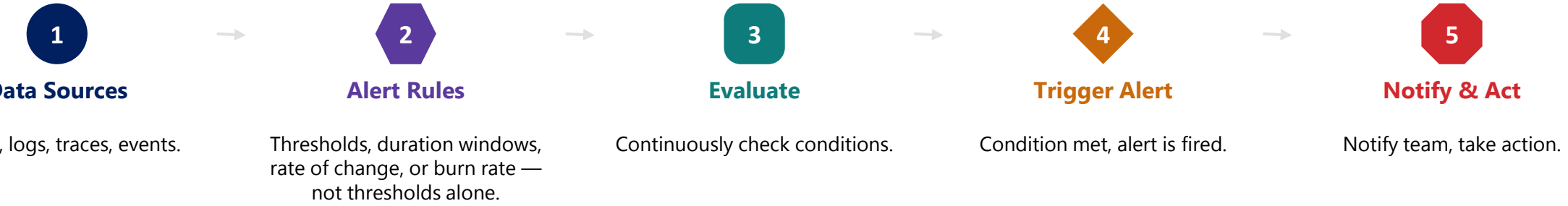
KEY POINTS

- Percentiles are computed per instance; percentiles-histogram exports buckets for accurate cross-instance aggregation.
- SLO buckets turn a timer into a count of requests under/over each threshold — ideal for burn-rate alerts.

KEY TAKEAWAY Configure percentiles per metric, not per instance average — use histogram buckets when aggregating centrally.

ALERTING FUNDAMENTALS

Alerting helps you proactively detect and respond to issues, notifying the right people at the right time.



ALERT SEVERITY LEVELS

LEVEL	EXPECTED ACTION	EXAMPLE
`Page`	Immediate human response	High error rate, service unavailable
`Ticket`	Investigation during working hours	High latency, high CPU usage
`Dashboard/event`	No immediate action (often not an alert at all)	Deployment successful, scale up

Severity (page/ticket/dashboard) and state (firing, acknowledged, resolved) are different dimensions — resolved is a state, not a severity level.
Avoid ambiguous "MTTR" — use explicit terms: mean time to detect, mean time to acknowledge, mean time to restore service, mean time to resolve root cause.
Every actionable alert needs an owner/team, runbook, dashboard link, query, and escalation route — not just a threshold. Notification channels: Slack, Email, PagerDuty, Teams, Opsgenie, Webhook, SMS.

KEY TAKEAWAY Good alerts map severity to action, separate state from severity, and always have an owner and a runbook.

EXPOSING ALERT SIGNALS WITH GAUGES

A Gauge can publish a derived value, like remaining error budget, for alerting rules to threshold against.

EXAMPLE: ERROR-BUDGET GAUGE (JAVA)

```
@Component
class SloMetrics {
    SloMetrics(MeterRegistry registry, SloTracker sloTracker) {
        Gauge.builder("slo.error_budget.remaining_ratio",
                sloTracker, SloTracker::remainingRatio)
            .description("Fraction of the 30-day error budget still available")
            .tag("service", "crm-api")
            .register(registry);
    }
}
```

KEY POINTS

- Alertmanager can threshold this gauge directly — page when remaining budget drops below a set fraction.
- Keep gauge tags low-cardinality (service, region) — never a per-customer or per-request identifier.

KEY TAKEAWAY Alerts are only as good as their input signal — a Gauge turns ‘error budget’ into something you can threshold on.

TRY IT YOURSELF — EXERCISE 5: BUILD AN ALERT RUNBOOK

Reinforces: the alert-rule anatomy (signal, threshold, severity, owner, runbook) you just saw.

STEP	WHAT TO DO
Goal	Write a mini runbook for an alert on create_failure_total
File	lab21-alert-runbook.md (in examples/module-21-exercises/)
Signal	create_failure_total rate exceeds a threshold for N minutes
Triage	Check /actuator/health first, then logs filtered by correlation id
CRM check	Reproduce create for a PROSPECT-shaped payload in non-prod
Reference	exercises/exercise-05-alert-from-failure-total.md

```
ALERT CrmCreateFailuresHigh
  IF rate(create_failure_total[5m]) > <threshold>
  FOR 5m            SEVERITY page          OWNER crm-team
  RUNBOOK notes/lab21-alert-runbook.md
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-alert-from-failure-total.md.

MONITORING DASHBOARDS

A dashboard should answer specific operational questions, not just display KPI tiles.

KPI TILE	EXAMPLE
`Total Requests (RPS)`	1,256
`p95 Response Time`	215 ms
`Error Rate`	0.35%
`Current 30-Day Availability`	99.95%

DASHBOARD TYPES & BEST PRACTICES

- A good dashboard answers: is the service healthy, are users impacted, what changed, where to investigate, and is capacity constrained?
- Organize by audience: executive/business, service overview, on-call diagnosis, capacity planning, release comparison.
- Show SLO target and error-budget remaining next to current availability — don't blend them into one 'SLA' number.
- For accessibility, mark thresholds with labels, icons, or patterns, not color alone.

Dashboards also track Active Users (e.g. 3,482). Common tools: Grafana, Kibana, Prometheus, Datadog, New Relic, CloudWatch.

KEY TAKEAWAY A good dashboard answers real questions for its audience — accessible status, not just a KPI wall.

TRY IT YOURSELF — EXERCISE 6: LAB 21 PREP CHECKLIST

Reinforces: everything from probes through alerting — one last check before Lab 21 begins.

STEP	WHAT TO DO
Goal	Confirm all five pre-lab artifacts exist before starting Lab 21
File	lab21-prep-checklist.md (in examples/module-21-exercises/)
Artifacts	Probes note, cardinality table, allow-list, metric TODOs, runbook
Boundary	Pre-lab only — prepare for Lab 21; don't complete the full lab
Pass / Fail	Pass if create_failure_total alert TODOs are filled in
Reference	exercises/exercise-06-lab21-prep-checklist.md

```
lab21-probes.md              # Exercise 1
lab21-cardinality-table.md   # Exercise 2
actuator-allowlist.md        # Exercise 3
lab21-metric-todos.md        # Exercise 4
lab21-alert-runbook.md       # Exercise 5
```

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-lab21-prep-checklist.md.

LAB OVERVIEW – OBSERVABILITY AND MONITORING

Add Spring Boot Actuator health checks and Micrometer metrics to the Northstar CRM.

WHAT YOU WILL LEARN

- Expose /actuator/health with liveness and readiness probes.
- Register Micrometer metrics (crm.customer.operations, crm.customer.operation.duration) for CRM create/get.
- Add a CrmReadinessIndicator (a labeled training simulation) that fails independent of liveness.
- Automate Actuator smoke tests and write a monitoring report.

LAB ENVIRONMENT

ITEM	VALUE
Application	Northstar CRM (single service)
Monitoring Stack	Actuator + Micrometer
Tools	curl, Maven, IntelliJ IDEA
Time	45 minutes

LAB OUTCOME

- You will expose Actuator health/readiness/liveness probes, register low-cardinality Micrometer metrics for CRM create/get, and document production exposure restrictions. This is a production-ready foundation for health and metrics — not full tracing, a metrics backend, dashboards, or alert delivery.

Prerequisites: Labs 19-20 complete (CRM create/get API, structured logging), JDK 21, Maven 3.9+, IntelliJ IDEA Community. Scope: this lab exposes /actuator/metrics only — Prometheus scraping is instructor-provided/demo, not built here.

KEY TAKEAWAY This lab builds the exact Actuator health and metrics foundation production monitoring depends on.

LAB TASKS AND DELIVERABLES

Add Actuator health probes and Micrometer metrics to the Northstar CRM.

#	TASK	KEY ACTIVITIES	FOCUS
1	Copy Lab 20 & Add Actuator	Copy lab20-crm to lab21-crm; add the Actuator starter.	Setup
2	Configure Health & Metrics	Expose health, info, and metrics via application.yml.	Configuration
3	Verify Liveness & Readiness	curl /actuator/health/liveness and /health/readiness.	Probes
4	Add CrmReadinessIndicator	Indicator flips readiness OUT_OF_SERVICE; liveness UP.	Readiness
5	Register Create/Get Metrics	Wire CustomerMetrics counters/timers by result tag.	Metrics
6	Drive Traffic & Metrics	POST/GET CUS-1001; compare metric values within the same running instance.	Verification
7	Automate Actuator Tests	ActuatorIT verifies liveness UP, readiness OUT_OF_SERVICE, and tagged metric increments.	Testing
8	Write Monitoring Report	Document metrics, alert idea, and exposure limits.	Documentation

SUCCESS CRITERIA

- Readiness can go OUT_OF_SERVICE (a labeled training simulation) while liveness stays UP; create-success counters increase after POST; ActuatorIT passes; monitoring report documents metrics, alert idea, and production exposure restrictions.
- Metrics: crm.customer.operations, crm.customer.operation.duration (tags operation=create|get, result=success|not_found|validation_error|failure — never customerId/correlationId/raw error text). ActuatorIT also checks tag-only increments and restricted exposure; timers record duration via finally even on failure; avoid a redundant duplicate counter.
- Production exposure: health limited, probes cluster-internal, metrics operator-only, env/beans/loggers restricted (scope: /actuator/metrics only — Prometheus is instructor-provided/demo). Report includes a programmatic tag-cardinality check and one actionable alert with signal, threshold, window, severity, owner, and runbook.

KEY TAKEAWAY You will have Actuator health probes and low-cardinality Micrometer metrics an operator can trust — with an explicit, honest scope.

PRODUCTION MONITORING: DEFINITION OF DONE

Production monitoring for an API is "done" when every item below holds true — prioritized by user impact, service objectives, critical dependencies, saturation, business workflows, then security/compliance.

1. User-facing SLIs are defined.
2. SLO targets and error-budget windows are documented.
3. Metrics use controlled, low-cardinality labels.
4. Liveness and readiness have distinct semantics.
5. Actuator exposure is restricted and authenticated.
6. Dashboards answer real operational questions.
7. Alerts are actionable, owned, and mapped to severity.
8. Runbooks exist for every actionable alert.
9. Logs, metrics, and traces share context (trace/correlation IDs).
10. Telemetry cost, alert tests, and post-incident reviews happen regularly.

KEY TAKEAWAY Monitoring is 'done' when this checklist holds true, not when a dashboard merely exists.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of observability and monitoring concepts.

1. A team adds a metric label customerId=CUS-1001. Why is this risky?

A. No effect on the metrics system

B. Creates high-cardinality series, exposes IDs

C. Makes dashboards load faster

D. Required for Prometheus to work

2. What is the primary purpose of distributed tracing?

A. Store application logs

B. Track a request across multiple services

C. Visualize server hardware metrics

D. Trigger alerts for log errors

3. Which library provides Spring Boot's metrics instrumentation facade?

A. Logback

B. Micrometer

C. Zipkin

D. Grafana

4. The database becomes temporarily unavailable. What should usually happen to the liveness probe?

A. Immediately fail and restart the pod

B. Stay UP; readiness should fail instead

C. No relationship to database health

D. Return HTTP 500 to callers

KEY TAKEAWAY Metrics, logs, and traces are the three core signals — Actuator and Micrometer are how you collect them.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. Which is an example of a good alert rule?

A. CPU usage > 50%

B. Error rate > 5% for 5 minutes

C. Memory used > 10%

D. Any 4xx error occurs

6. Which component visualizes metrics and dashboards?

A. Alertmanager

B. Prometheus

C. Grafana

D. Spring Cloud Sleuth

7. How do monitoring and observability differ?

A. They are the same thing

B. Monitoring = what; Observability = why

C. Observability only works in development

D. Monitoring replaces the need for logs

8. Why secure Actuator endpoints in production?

A. They are never needed

B. They expose sensitive operational information

C. They slow down the application

D. They only work with Prometheus

ANSWER KEY

- **1-B 2-B 3-B 4-B 5-B 6-C 7-B 8-B**

KEY TAKEAWAY Observability is not a one-time setup — continuously measure, learn, and improve your systems.

WEEK 2 REVIEW AND KEY TAKEAWAYS

This week, you built a strong foundation in enterprise Java development — organized into five connected themes, not a module-by-module list.

THIS WEEK'S THEMES

- ✓ **Build & dependency management (Modules 8-9)**
- ✓ **AI-assisted engineering (Modules 10-11)**
- ✓ **API & service-layer design (Modules 12-16)**
- ✓ **Testing strategy (Modules 17-19)**
- ✓ **Logging & observability (Modules 20-21)**

KEY TAKEAWAYS

- Build right from the start with a well-structured project and layered architecture.
- Design robust APIs with clear contracts and validated inputs.
- Test at every level: unit, integration, and UI.
- Observe and act — use logging, metrics, and tracing to detect issues early.
- Automate builds and use AI tools like GitHub Copilot responsibly.

WHAT'S NEXT? — Week 3 dives into Spring Framework, Data Access with JPA, Cloud-Native Development, and CI/CD Deployment.

KEY TAKEAWAY You are now equipped to build, test, and operate robust, scalable, and maintainable enterprise systems.

"You can't improve what you can't see. Good observability leads to faster insights and better outcomes."

MODULE 21 COMPLETE • WEEK 2 COMPLETE

API Observability and Monitoring

You can now build a practical, production-ready observability strategy using industry-standard tools and practices.